

AgentStage: Exploiting LLM Thinking for Data Staging in Scientific Agents

Izzet Yildirim, Xian-He Sun, Anthony Kougkas

Illinois Institute of Technology, Chicago, IL, USA

iyildirim@hawk.illinoistech.edu, sun@illinoistech.edu, akougkas@illinoistech.edu

Abstract—LLM agents on scientific data tasks typically spend most of their session wall time inside tool calls reading data from storage. Yet before each tool call, the agent reasons in a thinking phase, during which the storage layer sits idle and the next tool’s files begin to surface in the reasoning. Exploiting this is hard: the agent rarely names complete paths, so those files must be inferred and then staged before the tool fires. We present *AgentStage*, which reads the streaming reasoning and the agent’s filesystem probes to identify those files, then stages them into fast local storage with in-flight decompression so the tool’s reads land hot. We evaluate *AgentStage* on three curated I/O-bearing scientific tasks and one data-intensive task from each of three community benchmarks (MLE-bench, KramaBench, DSBench), across four reasoning models (three closed-weight, one open-weight). Under a paired trajectory-replay protocol that fixes each agent’s tool sequence and varies only cold versus staged execution, *AgentStage* delivers an average $1.74\times$ end-to-end speedup on the curated tasks, up to $2.34\times$ per run. On the three community benchmarks, *AgentStage* delivers up to $1.36\times$ speedup. The results show that thinking-phase staging helps when I/O share is high and prefetch completes within the thinking window, while compute-bound or many-file workloads see smaller gains. We show both conditions can be estimated before any staging run, so *AgentStage*’s benefit is predictable in advance. *AgentStage* adds negligible LLM-side overhead and needs no per-model or per-task tuning.

Index Terms—LLM agents, LLM thinking, scientific tasks, data staging, prefetch, HPC I/O, tiered storage

I. INTRODUCTION

Scientific data analysis is increasingly carried out by LLM agents that inspect datasets, write and run analysis code [1], [2], and revise from what they observe [3]–[7]. This makes the agent loop a data-accessing system, not only a sequence of prompts and responses. Each turn of this reason-then-act loop has two phases. In the *thinking phase*, the model plans the next tool call, weighing which files to read, which analysis to run, and how to structure its output [8], [9]. That plan then drives the *tool phase*, where the agent reads datasets from storage and computes over them [10]–[12]. The thinking phase recurs on every turn and structurally precedes each tool’s data access, an unavoidable component of the agent loop’s latency [13]–[15]. And as tasks and datasets scale [16], [17], the model reasons more before each tool call [18], stretching this thinking phase further.

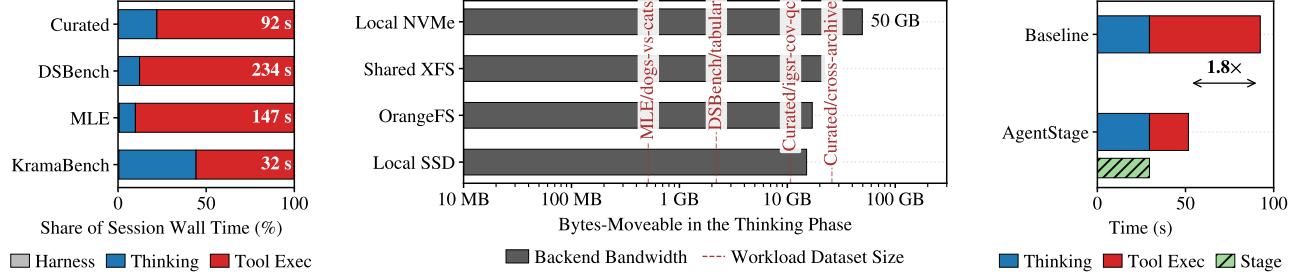
The thinking phase is also idle storage time, when no tool runs and no data moves. The data moves only in the tool phase that follows. Across our four benchmarks (Curated, DSBench [19], MLE-bench [20], and KramaBench [21]), tool execution occupies 53–93% of median wall time (Fig. 1a), much of it reading cold scientific data [22]–[24]. The reasoning

trace that fills the idle window reveals what to stage, because it decides the next tool’s reads and is emitted live. The agent’s own filesystem probes sharpen it further, with 96% of first tool calls across our experiments being directory listings (1s). And the window is wide enough to use. Within one thinking phase, the storage tiers we measure stage 15–50 GB into a hot tier (Fig. 1b), covering datasets up to tens of gigabytes.

The reasoning is visible and the staging is feasible, but nothing connects them. Prior observation and provenance work, such as AgentSight [25], PROV-AGENT [26], and OpenTelemetry [27], [28], records or models the reasoning stream. Another line of work acts on that stream, such as PASTE, which pre-executes the predicted next tool to hide its latency [13], and KV-cache reuse, which avoids recomputing model state [14], [29]. On the other hand, staging data into a hot tier has long been explored in HPC. For example, Markov prefetchers use access history [30], caches use recency [31], and burst buffers stage by a fixed workflow [32]. These rely on past behavior or a predefined plan, but an agent’s reads vary from session to session, named only in the reasoning as it streams. Staging from that signal remains open.

Turning this opportunity into a system is not simply copying files early. It creates three challenges. *Path extraction (C1)*: the signal is incomplete. The reasoning stream often names a semantic class (“the genomic VCF files in the project root”) or a partial glob (“the FOO_BAR_*.h5 series”) rather than a complete path. In our measured traces, the reasoning names no complete path in 58 of 72 runs (median zero per trace). *Subset selection (C2)*: the read set is smaller than the candidate set. A broad phrase such as “the training images” can match an entire workspace (22,500 files in one benchmark), while the tool opens only the batch the agent names. Over-staging wastes the thinking window, while under-staging leaves the tool on the cold path. *Data readiness (C3)*: the tool needs ready data, not just moved bytes. In 5 of 6 evaluated workloads, inputs must be decompressed, decoded, or paired with a sidecar index before a tool can read them. A BAM alignment, for instance, needs its companion BAI for indexed random access [33].

We present *AgentStage*, which turns the thinking phase into a prefetch window, reading the agent’s reasoning and filesystem probes to identify the next tool’s files and staging them into fast local storage, decompressed and ready, before the tool fires (Fig. 1c). Four components answer the three challenges in turn. An *auto-rule generator* mines each workload’s metadata into a rule library that a streaming *tiered detector* matches against the reasoning, recovering files from partial intent without manual



(a) Tool execution dominates every benchmark's wall time, leaving the thinking phase for prefetch.

(b) Data stageable during the thinking phase across the cold staging tiers. Dashed verticals mark workload dataset sizes (0.5–26 GB), most of which the thinking-phase staging budget already covers.

(c) Staging during the thinking phase keeps data hot, so the session finishes faster ($\approx 1.8\times$ here).

Fig. 1: Motivation: tool execution dominates session wall time (1a). The thinking phase alone can stage enough to cover typical workload datasets (1b). Overlapping that staging with the thinking phase recovers tool wall time (1c).

per-workload rule writing (C1). The detector folds in the agent's filesystem probes as they return, narrowing candidates to the set the tool will open (C2). A *compression-aware staging daemon* decompresses and co-stages sidecar indices while an `LD_PRELOAD` shim redirects the tool's reads to the staged copy, all without modifying the tool (C3). Under a paired trajectory-replay protocol that isolates the mechanism from model variance, across four reasoning models AgentStage delivers an average $1.74\times$ end-to-end speedup on the curated tasks with no per-model or per-task tuning. Because both conditions that govern the gain, cold-read share and whether prefetch finishes inside the thinking window, can be estimated before any staging run, the benefit is predictable in advance. This paper makes four contributions:

- **Thinking-phase staging-window characterization** across four reasoning models, showing it is long enough, against storage bandwidths, to fund prefetch (Section IV-C).
- **A tiered file-working-set detector** combining literal-path matching, auto-generated semantic rules from workload metadata, and filesystem-probe refinement, reaching median tier-1 byte recall 1.0 on the curated set (Section IV-B).
- **The AgentStage runtime**, a streaming capture proxy, auto-generated rule library, compression-aware staging daemon, and `LD_PRELOAD` path-rewriting shim (Section III).
- **An end-to-end staging evaluation** showing up to $2.34\times$ speedup on curated tasks and $1.36\times$ on community tasks (Sections IV-D, IV-E), and a two-factor model (I/O share and timeliness) for when staging helps (Section IV-F).

II. BACKGROUND AND RELATED WORK

A. Scientific LLM Agents and Reasoning Traces

LLM agents on scientific data tasks now follow a recurring loop: inspect a sandbox, draft analysis code, execute it, then revise from the returned observations [3]–[7]. Each iteration of that loop reads from shared scientific storage. Successive iterations of the same task can materialize different intermediates, take different failure paths, and re-read different files, so the workload is not the human-authored single-program access

pattern HPC I/O studies traditionally measure [34], [35], but a distribution of paths the agent could take through the filesystem.

Modern reasoning-tuned models emit thinking traces substantial enough to parse for intent [8], [9], [18], and the tool-call protocols that carry the parsed intent to the host harness have begun to standardize [36]. The agent's output-side interface (tool calls) has matured along the ReAct [37] and Reflexion [38] lines, while function-calling and tool-use training continue to extend the action space [39]. The combination produces a reliable streaming signal: thinking content arrives as tokens before any tool fires, and that content names, or names enough about, the files the agent's next tool will read.

B. Compute-Side Latency Hiding for Agent Loops

A growing body of work hides latency on the *compute* side of the LLM-tool loop. PASTE [13] pre-executes whole tools during the model's thinking phase using stable trajectory patterns mined from prior runs, reporting up to 48.5% task-time reduction on idempotent tools. KV-cache reuse systems extend the principle to the LLM side itself: vLLM [15] and Orca [40] define the modern serving primitives, SGLang [41] formalizes structured-program execution, and KVFlow [14] and LMCash/CacheBlend [29] generalize cache reuse across multi-agent workflows. Both lines attack the same wall-clock budget, the model's thinking time, but stop above the filesystem layer, hiding only compute. AgentStage acts on the data side: it consumes the same thinking phase PASTE waits in, but stages the files the agent's tool is about to read, leaving the tool itself to fire normally. It also reads a different signal, the in-flight thinking stream rather than historical trajectories, and because it speculates placement rather than computation, it needs no idempotent tools, works with any tool inside the managed POSIX file-access boundary, and pays a cold-cache miss rather than a discarded computation when it predicts wrong. The two compose.

C. Agent Observability and Provenance

A parallel line records or models what happens at the thinking-phase boundary without acting on it. AgentSight [25] captures syscalls from agent-launched code via eBPF, surfacing

the storage-side footprint without disturbing it. PROV-AGENT [26] models agent prompts, responses, and decisions as workflow provenance, so an error from one agent can be traced to its downstream effects. Orchestration-side telemetry, captured by OpenTelemetry’s GenAI semantic conventions [27] and tooling like Langfuse [28], exposes prompts, tokens, and tool calls at the LLM-side. AgentStage reuses the same classes of capture signal, but for storage-tier action rather than post-hoc analysis.

D. Data Fabric Vision and HPC Tiered Storage

The closest conceptual prior to AgentStage is the data-fabric vision of Giurgiu and Nidd [42], which argues that agentic workloads require storage that adapts to workload shape rather than the other way around. The deeper intellectual ancestor is Patterson’s TIP [43]: informed prefetching driven by hints in the source program. AgentStage is TIP-like, but the hints come from a streaming thinking stream rather than annotated source.

The HPC tiered-storage substrate AgentStage builds on includes node-local burst buffers and user-space distributed staging systems: Hermes [44], BurstFS [45], UnifyFS [46], GekkoFS [47], ProxyStore [48], HFatch [49], and DAOS [50]. AgentStage’s path-rewriting shim uses LD_PRELOAD rather than FUSE because FUSE adds per-syscall overhead that becomes load-bearing for metadata-heavy agent workloads [51].

III. SYSTEM DESIGN

A. Architecture Overview

AgentStage maps each of the three challenges identified in Section I onto a system component (Fig. 2). C1 is handled by the *auto-rule generator* and the *tiered detector* (2) together: the generator mines partial-intent patterns from workload metadata into a per-workload rule library, and the detector scores streaming thinking content against that library to produce candidate paths. C2 is handled by combining the detector’s streaming output with the filesystem-probe responses the agent itself materializes during reasoning. The detector refines candidates in flight rather than committing on first signal. C3 is handled by the *compression-aware staging daemon* (3) and an LD_PRELOAD path-rewriting shim (4): the daemon copies cold-tier files into the local hot tier with optional decompression, and the shim redirects the agent’s `open()` syscalls to the staged hot copy when the tool fires. A *capture proxy* (1) intercepts the LLM stream to feed events to the detector without buffering and without measurable LLM-side latency. Components (1)–(3) operate during the LLM-side thinking phase, and (4) activates when the tool fires.

B. Capture Proxy

The capture proxy is an in-process SDK wrapper, not an HTTP proxy: AgentStage wraps each provider’s streaming client, forwarding each event to the harness unchanged while a per-event hook fires the detector against the accumulated thinking content. The hook runs at the granularity of a single content delta, so the detector sees intent the moment the model emits it, with no buffering and no replay. It has one performance pitfall: re-running the rule library over the whole

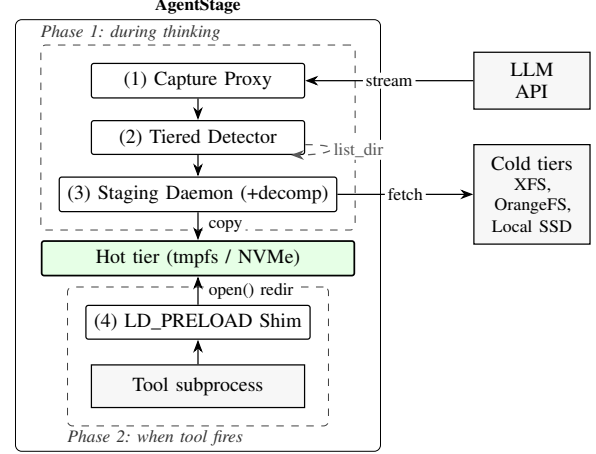


Fig. 2: AgentStage architecture. The capture proxy (1) consumes the LLM stream. The detector (2) matches it and refines via filesystem-probe responses. The staging daemon (3) copies files cold-to-hot with optional decompression. When the tool fires, the LD_PRELOAD shim (4) redirects the subprocess’s `open()` calls to the staged copy. The hot tier bridges the phases: written during thinking, read at tool fire.

accumulated thinking text on every delta is quadratic in thinking length. AgentStage instead matches each unfired rule against only the new delta, keeping per-event cost flat in rule count. Pass-through is byte-identical when the detector is disabled, and Section IV-C measures the LLM-side overhead.

C. Tiered Detector

The detector has two layers stacked behind a single rule library. The literal-path layer scans the accumulated thinking text for substrings that look like file paths, gated by a unique-basename check against the workspace prior. It is high-precision and low-recall by design: when it fires it almost always names a real input file, but it fires rarely because the model seldom commits to literal paths (Section I). The semantic-class layer fills the gap with named-class regex rules whose target sets are sized by a workspace prior, a map from class names to the matching workspace paths.

The rule library is generated by the *auto-rule generator* from two pieces of workload metadata, the task-instruction text and the workspace-prior bucket keys, as three rule families. (i) *Per-instance rules*. Each bucket key carries a class prefix and an instance (`detector_nrcal`, `sample_HG00096`, `band_08`). The generator splits the two and emits one rule per bucket whose regex matches the instance in the forms a model is likely to write: the class label joined to the instance by an optional space, hyphen, or underscore (`band 08`, `band-08`, `band08`), with a leading-zero-stripped variant for numeric instances (`band 8`); the bare token (`nrcal`); a suffix-only variant for hyphenated names (`sub-Cori` also matching `Cori`); and a path-component form (`/122/`) for numeric day, month, year, and session instances. (ii) *General rules*. The generator

scans the instruction for scientific format tokens (BAM, FITS, NetCDF, CSV) and seeds format-aware first-inspect and all-files rules over the workload’s aggregate buckets. (iii) *Output-file rules*. Filenames named in the instruction, such as `integrity_manifest.csv`, become rules over the corresponding output buckets. Buckets a general rule already covers are excluded from the per-instance family. On `jwst-nircam` this yields one rule per NIRCcam detector, matching `nrcal` through `nrcb4` over the FITS files of that detector, each target set small enough for tier-1. The generator runs once per workload and is deterministic: the same workload yields the same rule set. *The same generator applies to every benchmark*, reaching median tier-1 byte recall 1.0 (Section IV-B) without manual per-workload rule writing or per-model tuning.

Activated rules emit candidate path sets. The detector classifies each rule into a tier by its target-set size. Tier-1 covers small target sets (≤ 20 files), staged eagerly. With per-instance decomposition (e.g., `igsr-cov-qc`), the union of fired tier-1 rules dispatches many files while each stays small. Tier-2 covers medium sets (≤ 200 files), staged opportunistically, and tier-3 covers larger sets (> 200 files), deferred to explicit demand. The tiering matters most on the high-fanout case: on `dogs-vs-cats` (22,500 files), AgentStage defers the monolithic image bucket to tier-3 and stages it on demand as the agent names batches, rather than overfetching the whole workspace.

The detector refines candidates in flight using the filesystem-probe responses the agent itself materializes during reasoning (Section I). Each probe result narrows the candidate set, so subsequent reasoning matches against the now-known subset rather than the prior. These cheap probes differ from the first expensive shell tool, whose cold reads dominate session wall time and bound the staging window. A large initial candidate set therefore shrinks as the agent walks the filesystem.

D. Staging Daemon and Path-Rewriting Shim

The staging daemon receives candidate-path messages from the detector over an in-process queue and copies the named files from their cold tier into the hot tier. AgentStage supports parallel, networked, and local cold backends. The Ares testbed exercises three of them (Local SSD, OrangeFS over FUSE, and Shared XFS over a network-mounted POSIX export). The hot tier is tiered: `tmpfs (/dev/shm)` is the default for fastest reads, with a dedicated Local NVMe mount as the fallback when `tmpfs` capacity is exceeded. In-paper experiments use `tmpfs` (the largest measured workload is 26 GB, well within RAM). The hot tier is capacity-bounded, evicting least-recently-used staged files when a copy would exceed the budget, while protecting copies in flight. Each cold tier is named by a *cold-root prefix*. Staging materializes a hot copy at the corresponding *hot-root prefix*, preserving the relative path under the root.

The daemon is compression-aware. Scientific data often ships compressed (`.gz`, `.zst`, `.tar.gz`). When a rule indicates the tool reads decompressed bytes (e.g., `pandas.read_csv` on a `.csv.gz`), AgentStage decompresses at stage time and the shim returns the decompressed hot copy on `open()`. When the rule indicates the tool reads the compressed

Algorithm 1 AgentStage on-stream detect→stage policy.

```

1: Phase 1: detect (per event  $\delta$ : thinking, text, or tool result):
2: accumulated  $\leftarrow$  accumulated +  $\delta$ 
3: for all rule  $R \in$  rules with  $R.name \notin$  fired do
4:   if REGEX.SEARCH( $R.regex$ , accumulated) then
5:     fired.ADD( $R.name$ )
6:      $C \leftarrow$  workspace_prior[ $R.class$ ]  $\cap$   $R.target\_set$ 
7:     if TIER( $|C|$ )  $\in \{1, 2\}$  then
8:       for all path  $p \in C$  do
9:         stage_queue.DISPATCH( $p$ , MODE( $p$ ))
10:      end for
11:    end if (tier-3 deferred to explicit demand)
12:  end if
13: end for
14: Phase 2: stage (daemon, per ( $p, mode$ )):
15: rel  $\leftarrow p -$  cold_root; hot_p  $\leftarrow$  hot_root + rel
16: if mode = DECOMP then
17:   hot_p  $\leftarrow$  STRIP_EXT(hot_p); DECOMPRESS(cold_root + rel, hot_p)
18: else
19:   COPY(cold_root + rel, hot_p)
20: end if
21: staging_state[p]  $\leftarrow$  (STAGED, hot_p)

```

file directly (e.g., a `gzip -d` invocation), the file stages unchanged. Tools are never modified. For primary files paired with a *sidecar index* (a small companion file required for random access, like a BAM’s BAI index [33] or Zarr’s `.zarray` metadata [52]), the daemon co-stages the sidecar.

The path-rewriting shim closes the loop. It is a small C library loaded into agent-spawned subprocesses via `LD_PRELOAD`, interposing on `openat`, `stat`, `access`, and their variants. When the call’s path lies under a managed cold root *and* a hot copy is already staged, the shim rewrites the path to the hot equivalent and lets the call through. Otherwise, the call falls through to the cold tier unchanged. We chose `LD_PRELOAD` over FUSE because the agent’s tool issues hundreds of `stat/openat` calls during script startup, where FUSE’s per-syscall overhead [51] would dominate, and `LD_PRELOAD` needs no kernel modules or root access.

Algorithm 1 summarizes the two-phase pipeline end-to-end. Phase 1 runs in the capture proxy per stream event and dispatches candidate paths classified by tier. Phase 2 runs in the staging-daemon workers, materializing a hot copy and decompressing in flight when the cold file is compressed.

E. Implementation

AgentStage is a Python package plus a single C shim.¹ The proxy, detector, and staging daemon are 6,700 lines of Python running in the agent harness process, and the shim is 560 lines of C loaded into agent-spawned tool subprocesses. The daemon is a thread pool in the harness address space, and the filesystem is its only channel to the shim: each file is copied to a temporary path and then atomically renamed into place, so the shim never observes a partial copy. The shim interposes only on path-taking calls, since `read` and `mmap` operate on descriptors and follow the redirect once `openat` returns a hot one. Writes pass through to the cold tier unchanged, leaving the hot tier read-only from the agent’s perspective. Integration requires

¹AgentStage source code is available at <https://github.com/izzet/agentstage>

only replacing the provider’s SDK client with AgentStage’s drop-in equivalent and exporting `LD_PRELOAD` for the shim.

IV. EVALUATION

A. Experimental Setup

1) *Testbed*: Experiments run on the IIT GRC Ares cluster (per-node: dual Intel Xeon, 48 GB RAM, Local NVMe and SATA SSD, 40 GbE). AgentStage targets three cold-tier backends slow-to-fast: Local SSD, OrangeFS (FUSE over node-local NVMe), and Shared XFS. OrangeFS is the cold tier for all reported cells. The hot tier is tmpfs (`/dev/shm`) primary with Local NVMe fallback. Per-tier read bandwidths are tmpfs 4494 MiB/s, Local NVMe 1676 MiB/s, Shared XFS 743 MiB/s, OrangeFS 576 MiB/s, and Local SSD 511 MiB/s.

2) *Workloads*: We evaluate AgentStage on a *Curated* set of three I/O-bearing tasks we author from open scientific datasets, plus one data-intensive task from each of three community benchmarks (MLE-bench [20], KramaBench [21], DS-Bench [19]) chosen to exercise the staging mechanism in its operating regime (Table I). The community tasks provide end-to-end evidence on workloads not authored alongside the detector.

The curated tasks span three scientific domains. `igsr-cov-qc` is a data-integrity manifest over BAM alignments from the IGSR 1000 Genomes archive [33], [53], `jwst-nircam` over JWST coadd FITS products, and `cross-archive` adds Sentinel-2 NDVI GeoTIFF scenes for a larger, two-domain working set. The community cells cover image preprocessing (MLE-bench `dogs-vs-cats`), heliophysics inventory (KramaBench `astronomy-inventory`), and tabular integrity (DSBench `tabular-oct-2021`). Across the six tasks, file counts span 3 to 22,500 and dataset sizes 506 MB to 26 GB.

3) *Models*: We test four reasoning models (Table II): three closed-weight (Claude Haiku 4.5, Sonnet 4.5, Gemini 2.5 Flash) served over vendor streaming APIs, and the open-weight Qwen3.6-27B, self-hosted with vLLM on NCSA DeltaAI [54].

4) *Methodology*: Every baseline-vs-staged comparison runs against verified-cold OrangeFS caches. Before each run we evict the workload dataset directory via `posix_fadvise(POSIX_FADV_DONTNEED)` and verify via `mincore` that the resident page fraction is at most 0.05 before timing begins. Without this discipline, OS page cache from prior runs masks the cold-tier baseline.

Each cell is measured via a paired trajectory-replay protocol. We record one live session per (benchmark, model, task, rep), capturing the tool-call order and the timestamped thinking and text deltas. Each trajectory is replayed twice on the same verified-cold cache: a *cold replay* with the AgentStage shim and stager disabled, and a *staged replay* with both running live. In both, the deltas are paced back at their original wall-clock timing so staged-mode prefetch races LLM streaming as in a live session. Per-cell session speedup is reported as `cold_replay_elapsed/staged_replay_elapsed`. Holding the trajectory fixed across modes isolates mechanism contribution from LLM streaming variance. AgentStage’s capture proxy (Section III) timestamps each stream event and tool invocation,

TABLE I: Workload characterization: three curated tasks plus one data-intensive task from each of MLE-bench, KramaBench, and DSBench, in the mechanism’s operating regime.

Bench	Task	Domain	Files	Total
Curated	<code>igsr-cov-qc</code>	Genomics	153	10.7 GB
Curated	<code>jwst-nircam</code>	Astronomy	96	11.3 GB
Curated	<code>cross-archive</code>	Multi-domain	176	26.0 GB
MLE-bench	<code>dogs-vs-cats</code>	Image preproc.	22 500	537 MB
KramaBench	<code>astronomy-inventory</code>	Heliophysics	1 538	506 MB
DSBench	<code>tabular-oct-2021</code>	Tabular regr.	3	2.3 GB

TABLE II: Reasoning models evaluated, spanning three provider families, open and closed weights, and compact to frontier classes. All stream the reasoning the detector consumes.

Model	Class	Weights	Hosting
Claude Sonnet 4.5	Frontier	Closed	Vendor API
Claude Haiku 4.5	Compact	Closed	Vendor API
Gemini 2.5 Flash	Compact	Closed	Vendor API
Qwen3.6-27B	Mid-size	Open	Self-hosted (DeltaAI)

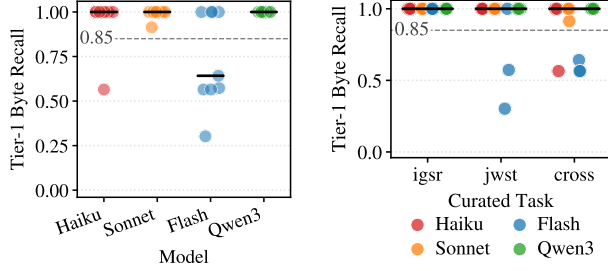
so we log the per-turn wall-time breakdown behind Fig. 1a, submission validity, and the staging daemon’s prefetch report.

B. Detection Accuracy

We score the tiered detector on the 36 sessions of the curated set (`igsr-cov-qc`, `jwst-nircam`, `cross-archive` $\times$ 4 models $\times$ 3 reps). For each session we replay the per-turn thinking and visible-text plus prior-turn tool results through the detector (the signals the live detector consumes before each tool fires) and compare tier-1’s staged set against the trajectory’s full read set. The auto-rule generator produces a per-workload rule set applied across all models without per-model tuning. The generator consumes only pre-session workload metadata (task instruction and workspace inventory keys). It does not consume model traces, baseline or staged outcomes, first-inspect ground truth, or evaluation-session text.

Fig. 3 shows the per-session tier-1 byte recall (recall 1.0 = every file the agent’s session reads is staged) scored against the workload’s full read set. Median tier-1 byte recall is 1.0 for Haiku, Sonnet, and Qwen3, and 0.64 for Gemini Flash. The Flash gap reflects a reasoning-style difference: Flash sessions on the per-detector and per-scene curated tasks enumerate fewer workspace tokens before the first tool fires, so the auto-generator’s per-instance rules cover a smaller portion of the full read set. The auto-generator’s per-workload output is identical across models on a given workload and generalizes across three provider families without per-vendor tuning.

Recall alone does not establish selectivity. These tasks read essentially their entire workspace, so a policy that staged every file would also reach recall 1.0. Two measurements bound what the detector stages. *Path-validity* precision is 1.0 across all four models: no dispatched file falls outside the workload’s valid bucket structure. Over-fetch, bytes staged but never read, stays under 8% of dataset bytes on bandwidth-bound workloads where staging dominates session time, rising only on low-I/O-share workloads where the wasted copy costs little wall time.



(a) Per-model recall: median 1.0 (b) Per-task recall: all models for Haiku, Sonnet, and Qwen3, cover *igsr* fully, while Flash falls and 0.64 for Flash. on *jwst* and *cross*.

Fig. 3: Tier-1 byte recall on the 36 curated cells (3 tasks $\times$ 4 models $\times$ 3 reps), scored against the full session read set. Per-instance rules target only files within the workload’s bucket structure, so path-validity precision is 1.0 (no out-of-bucket dispatch). Dashed line marks the 0.85 recall threshold.

TABLE III: Detection ablation (curated set, 36 cells). Mean tier-1 byte recall and detection-firing rate for: (H) task-description rules, (A1) auto-generator from thinking only, and (A2) auto-generator plus filesystem probes. Both additions are needed.

Variant	Mean Recall	Sessions w/ Detection
(H) Task-description rules	0.00	9/36
(A1) Auto, thinking only	0.30	21/36
(A2) Auto + filesystem probes	0.88	35/36

Across the full 72-cell pool, agent reasoning names no complete file path in 58 of 72 runs (median zero per trace), confirming that the auto-generated rules, not literal-path matching, carry detection. Two ablations isolate the auto-generator’s and filesystem-probe refinement contributions (Table III). (H) Task-description-based rules (3–5 per task, with no workspace-inventory traversal) target aggregate buckets that classify as tier-2/tier-3 by target-set size and never stage eagerly: only 9 of 36 sessions fire any eager detection and mean recall is 0.00. (A1) The auto-generator without prior-turn probe responses decomposes the workspace into per-instance rules (per-sample, per-detector, per-scene), each fitting tier-1. Mean recall rises to 0.30 and detection fires on 21 of 36 sessions: rules trigger whenever the agent’s reasoning names a workspace token, but sessions whose visible reasoning stays abstract leave per-instance rules silent. (A2) Adding the agent’s filesystem-probe responses universally surfaces per-instance tokens in tool-result text: mean recall reaches 0.88 and detection fires on 35 of 36 sessions. The combination of automatic inventory-derived rules and filesystem-probe refinement is what brings the detector to its headline configuration.

C. Detection Latency

AgentStage’s mechanism converts the thinking phase into a prefetch window only if the detector can identify the next files to stage early enough to issue copies before the agent’s tool fires. We characterize this lead time on the curated set by

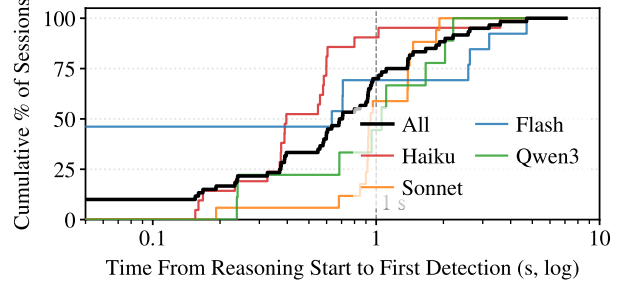


Fig. 4: The detector fires within 1 s of reasoning in 70% of sessions, opening the thinking phase as a prefetch window. Time from the first thinking delta to the first activation matching the first-inspect target, on per-delta timestamps over the curated sessions. Median 0.70 s, dashed line at 1 s.

replaying each session’s per-turn thinking deltas with wall-clock timestamps, running the detector live, and recording the time from the agent’s first thinking delta to the first rule activation whose target files intersect with the workload’s first-inspect ground truth. The result is the latency the system must subtract from each session’s thinking phase before staging can begin.

Fig. 4 shows the cumulative distribution across the 60 recorded curated sessions (all three tasks across four models and reps), a superset of the 36 cells (three reps each) replayed for the costlier true-cold speedup of Section IV-D. Median latency is 0.70 s, with 70% of sessions reaching the first correct fire within 1 s and 88% within 2 s. Per-model distributions: Haiku median 0.40 s (p95 1.02 s), Sonnet median 0.94 s (p95 1.92 s), Flash median 0.63 s (p95 4.71 s), Qwen3.6 median 1.06 s (p95 2.21 s). The longer-tail Flash and Qwen3 sessions reflect early thinking content that frames the task before naming workspace paths. They extend the latency before staging starts, not the eventual detection quality (Fig. 3).

Absolute latency only matters against the window it consumes, and that window itself ranges from 2 to 106 s across these sessions. Normalized per session, detection consumes a median 2.9% of the window, leaving 97% of it usable for staging, and 83% of sessions retain more than 90%. The capture proxy feeding the detector imposes no observable cost on the LLM critical path either: $p99$ overhead ratio $\leq 1.001\times$ over the no-proxy baseline on a 10,638-event replay over 477 sessions, with maximum event skew 4.5 ms.

D. Session Speedup

Across the 36 curated cells (4 reasoning models $\times$ 3 tasks $\times$ 3 reps), AgentStage delivers an arithmetic-mean session speedup of $1.74\times$ (geometric mean $1.70\times$, 95% CI ± 0.11). Per-task means are $1.58\times$ on *igsr-cov-qc*, $1.79\times$ on *jwst-nircam*, and $1.84\times$ on *cross-archive*. Per-cell speedups span $1.00\times$ to $2.34\times$, the maximum on Gemini Flash $\times$ *jwst-nircam*. Fig. 5 shows per-(task, model) means as bars with per-cell reps overlaid as dots.

Per-model means span a narrow band: Qwen3.6-27B $1.88\times$, Gemini Flash $1.83\times$, Haiku $1.73\times$, Sonnet $1.52\times$ (each $n = 9$

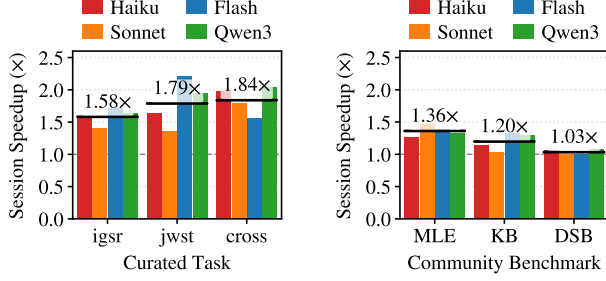


Fig. 5: Session speedup on the curated tasks, by model. Staging delivers consistent gains across tasks and models; per-task means 1.58 to 1.84 $\times$.

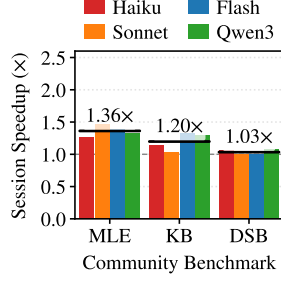


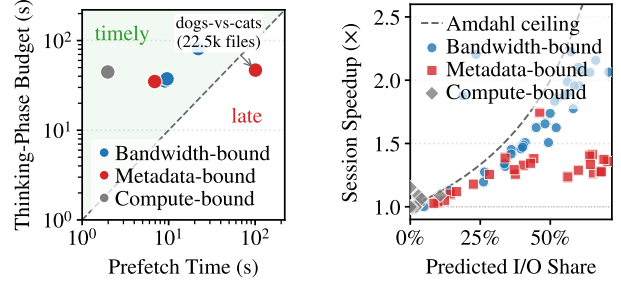
Fig. 6: Session speedup on community benchmarks, by model. Transfer is positive to near-neutral (1.03 to 1.36 $\times$), matching regimes in Fig. 7.

across the three curated tasks). The 0.36 $\times$ spread between fastest- and slowest-mean models is far smaller than the 1.34 $\times$ per-cell span, indicating that speedup magnitude is driven by task structure (per-instance bucket decomposition, shell-time share, prefetch coverage) rather than by model. The open-weight Qwen3.6-27B lands at the same magnitude as the closed-weight models, supporting that the mechanism is not specific to any provider family. Gemini Flash’s strong mean despite its lower median detection recall (Fig. 3) is consistent: a few large sequential files dominate read time on these bandwidth-bound tasks, so the cells where Flash covers those files post large speedups even when its median file recall is lower.

Two robustness checks frame the result. *Trajectory validity*: the trajectories we replay are live sessions that completed and submitted the task, so each speedup is measured over a real task completion rather than a truncated run. *Rep variance*: within-cell session-speedup standard deviation stays below 0.15 $\times$ on the majority of cells, so per-cell speedups represent stable trajectory-level measurements rather than rep-stochastic noise. The session-level distribution is the wall-clock projection of the mechanism results established earlier in this section: tier-1 byte recall remains at 1.0 on the median session (Fig. 3), and detector activation completes within 0.70 s of the first thinking delta (Fig. 4), recovering more than 97% of the thinking phase as a prefetch window.

E. Generalizability to Community Benchmarks

We test transferability on one data-intensive task from each of three community benchmarks: MLE-bench *dogs-vs-cats* (image preprocessing, 22,500 files), KramaBench *astronomy-inventory* (heliophysics archive, 1,538 files), and DSBench *tabular-oct-2021* (tabular regression, 3 files), each evaluated under the same paired trajectory-replay protocol across 4 models $\times$ 3 reps (36 community cells total). Per-benchmark arithmetic-mean session speedups are 1.36 $\times$ on MLE-bench, 1.20 $\times$ on KramaBench, and 1.03 $\times$ on DSBench. Per-model means stay within a



(a) Prefetch time vs. thinking-phase budget. Points above the diagonal are timely: prefetch finishes within the thinking window.

(b) Observed speedup vs. predicted I/O share for 72 cells, with the Amdahl ceiling $1/(1-s)$. I/O share sets the ceiling.

Fig. 7: When thinking-phase staging pays off. I/O share sets the ceiling (b); timeliness sets what is realized (a). Bandwidth-bound workloads satisfy both, reaching 91% of the ceiling; MLE-bench’s corpus is timeliness-limited (prefetch cannot finish in time); compute-bound workloads lack headroom.

narrow band across the community tasks (Fig. 6): Gemini Flash 1.24 $\times$, Qwen 1.24 $\times$, Haiku 1.15 $\times$, and Sonnet 1.17 $\times$.

The three community benchmarks span the operating regimes that Section IV-F characterizes, which produces the spread. Unlike the curated manifests, whose per-instance decomposition eager-stages every file via tier-1, the community workloads exercise the daemon’s other paths. MLE-bench (*dogs-vs-cats*, 22,500 files) is metadata-bound and timeliness-limited: its monolithic image bucket routes to tier-3 on-demand staging, dispatched as the agent materializes file-naming intent in tool-result feedback, and its many server-serialized opens do not complete inside the thinking window, so staging hides only part of the I/O for a 1.36 $\times$ speedup. KramaBench (*astronomy-inventory*, 1,538 files) is metadata-layout but small, so its I/O share is modest and the prefetch stages in time, giving 1.20 $\times$. DSBench (*tabular-oct-2021*, 3 files) is compute-bound: the session is dominated by model fitting, so its I/O share is near zero and near-unit speedup (1.03 $\times$) is expected rather than a staging failure. Community transfer is thus positive to near-neutral, and the two-factor model predicts which case each benchmark falls into.

F. When Staging Pays Off

The session speedups of Sections IV-D and IV-E vary widely across workloads, and a two-factor model explains the spread with no per-cell fitting. Staging can only hide the cold-tier I/O a session actually performs, so the speedup ceiling is set by the *I/O share*: the fraction of session wall time spent reading the cold tier. We predict this share a-priori from data layout: an I/O cost model calibrated once per cold tier (sequential read bandwidth and per-file open latency, from microbenchmarks) is applied to a workload from its file count and byte size alone, using no staged-run timings. The model recovers the measured cold-read time across the six workloads at $R^2 = 0.998$.

Whether a workload realizes that ceiling is then set by *timeliness*: whether the prefetch finishes inside the thinking-phase window. Speedup follows only when both conditions hold.

Fig. 7b plots observed speedup against predicted I/O share for all 72 cells, with the Amdahl ceiling $1/(1 - \text{io_share})$. Bandwidth-bound workloads (the curated archive-retrieval tasks) carry a high I/O share and average $1.74\times$, reaching 91% of their ceiling. Compute-bound DSBench reads only three files, so its I/O share is near zero and near-unit speedup ($1.03\times$) is the ceiling itself. The metadata-bound workloads sit between, with an I/O cost dominated by per-file opens (file count times open latency): modest for KramaBench (1,538 files, $1.20\times$) but large for MLE-bench (tens of thousands of files).

Whether that ceiling is realized is the second factor, *timeliness* (Fig. 7a). The prefetcher runs four-wide at 1179 MB/s, $3.4\times$ the agent’s single-stream cold read, so the curated datasets stage in 9 to 22 s, well inside the 35 to 81 s thinking budget, and realize 84 to 99% of their ceiling. The one exception is MLE-bench: staging its tens of thousands of server-serialized opens takes over 100 s and overruns the 47 s budget, so staging hides only part of the I/O and the workload lands well below its high ceiling ($1.36\times$). Timeliness, like I/O share, follows a-priori from the prefetch cost and the measured thinking budget, the window the detector exploits for early activation (Section IV-C). Both factors are known before any staging run, so a workload’s regime and its ceiling follow without staging it first.

V. DISCUSSION

Bandwidth bounds. The session-time speedup is bounded by the cold-tier I/O share of the tool phase. On Ares the cold-to-hot read bandwidth ratio is $7.8\times$ to tmpfs and $2.9\times$ to the NVMe fallback. Cells dominated by per-tool compute (tabular regression on DSBench, metadata aggregation on KramaBench) reach a realized session ceiling below that I/O share even when detector and daemon succeed: the wall-clock gain is the product of mechanism success and I/O share (Section IV-F). The same model explains why curated cells, where bulk file reading dominates per-tool work, post a higher ceiling and the larger observed gains.

Generalizability. Cells with no prefetch dispatch are a rule-coverage gap, not detector failure: the agent reasoned without naming any path-extracting tokens, and the same boundary applies to the detection-latency tail (p95 4.71 s on Gemini Flash and 2.21 s on Qwen3.6). The LD_PRELOAD shim is inherited across fork and execve, propagating to multi-process tool stacks (Python multiprocessing, and MPI launchers when the launcher forwards the environment). Generalization to substantially different workload classes (workflow-DAG agents) is out of scope here.

Multi-agent deployments. Concurrent runs against shared data make the staging cache a cross-agent resource. On one node, co-located agents could reuse a single tmpfs hot-tier copy via shared memory. Across nodes, distributed-memory pools could hand off data when one agent’s output becomes another’s input. The research question is cross-cohort amortization of

cold-fetch cost, with a global rate-limit preventing contention from degrading the cold-tier baseline.

Long-horizon agents. Long-horizon agentic loops generate new file-naming intent on each round of exploration. The staging tier is finite, so it becomes a working set requiring priority-weighted eviction beyond the current LRU, and dynamic reallocation as the agent’s reasoning shifts focus. A bidirectional detector-daemon channel can signal stale targets alongside fresh ones, so capacity tracks current intent rather than session-cumulative dispatch. This is contingent on provider passthrough of thinking content on subsequent turns. Without it, the per-event detector loses its signal after turn one.

VI. CONCLUSION

AgentStage demonstrates that the LLM thinking phase carries enough actionable intent to stage data ahead of the next tool call. A streaming capture proxy, an auto-generated rule library, a compression-aware daemon, and an LD_PRELOAD shim turn that intent into hot-tier reads. On the targeted regime of three curated I/O-bearing tasks (36 cells across four reasoning models), the median session’s detector identifies the next tool’s files within the first second of reasoning, and arithmetic-mean session speedup is $1.74\times$ with per-cell speedups up to $2.34\times$. Two a-priori factors, the I/O share and prefetch timeliness, explain when staging delivers speedup, and the mechanism transfers positively to community benchmarks (MLE-bench, KramaBench, DSBench) with per-benchmark mean 1.03 to $1.36\times$. AgentStage applies without per-model tuning across four reasoning models from three provider families.

The mechanism opens the data side of agentic latency-hiding, complementary to compute-side techniques such as speculative tool execution and KV-cache reuse. Its leverage scales with the cold-to-hot bandwidth gap, so production parallel filesystems and object stores are a natural next target. Several research directions follow: cross-agent caching for concurrent multi-agent deployments, eviction and priority policies for long-horizon agentic loops, and admission control for shared cold tiers under multi-tenant contention. As scientific agents take on longer horizons and larger data volumes, the thinking-phase prefetch window becomes a load-bearing primitive for the next generation of scientific agents.

ACKNOWLEDGMENT

This material is based upon work supported in part by the National Science Foundation (NSF) under Awards OAC 2411318 and OAC 2313154. This work used DeltaAI at the National Center for Supercomputing Applications (NCSA) through allocation CIS250329 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program [55], which is supported by NSF grants #2138259, #2138286, #2138307, #2137603, and #2138296. DeltaAI is an advanced computing and data resource supported by NSF (award OAC 2320345) and the State of Illinois, and a joint effort of the University of Illinois Urbana-Champaign and NCSA.

REFERENCES

- [1] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.
- [2] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [3] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, no. 7992, pp. 570–578, Dec. 2023.
- [4] A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller, "Augmenting large language models with chemistry tools," *Nature Machine Intelligence*, vol. 6, no. 5, pp. 525–535, May 2024.
- [5] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu, C. Zhang, D. Li, J. Chen, J. Zhang, J. Wang, L. Zhang, L. Zhang, M. Yang, M. Zhuge, T. Guo, T. Zhou, W. Tao, R. Tang, X. Lu, X. Zheng, X. Liang, Y. Fei, Y. Cheng, Y. Ni, Z. Gou, Z. Xu, Y. Luo, and C. Wu, "Data Interpreter: An LLM Agent for Data Science," in *Findings of the Association for Computational Linguistics: ACL 2025*. Vienna, Austria: Association for Computational Linguistics, 2025, pp. 19 796–19 821.
- [6] C. Lu, C. Lu, R. T. Lange, Y. Yamada, S. Hu, J. Foerster, D. Ha, and J. Clune, "Towards end-to-end automation of AI research," *Nature*, vol. 651, no. 8107, pp. 914–919, Mar. 2026.
- [7] M. Sun, R. Han, B. Jiang, H. Qi, D. Sun, Y. Yuan, and J. Huang, "LAMBDA: A Large Model Based Data Agent," *Journal of the American Statistical Association*, vol. 121, no. 553, pp. 1–13, Jan. 2026.
- [8] M. Bosma, E. Chi, B. Ichter, Q. V. Le, D. Schuurmans, X. Wang, J. Wei, F. Xia, and D. Zhou, "Chain-Of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems 35*. New Orleans, Louisiana, USA: Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022, pp. 24 824–24 837.
- [9] Y. Cao, T. Griffiths, K. Narasimhan, I. Shafran, S. Yao, D. Yu, and J. Zhao, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," in *Advances in Neural Information Processing Systems 36*. New Orleans, Louisiana, USA: Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023, pp. 11 809–11 822.
- [10] Z. Chen, S. Chen, Y. Ning, Q. Zhang, B. Wang, B. Yu, Y. Li, Z. Liao, C. Wei, Z. Lu, V. Dey, M. Xue, F. N. Baker, B. Burns, D. Adu-Ampratwum, X. Huang, X. Ning, S. Gao, Y. Su, and H. Sun, "ScienceAgentBench: Toward Rigorous Assessment of Language Agents for Data-Driven Scientific Discovery," 2024.
- [11] X. Hu, Z. Zhao, S. Wei, Z. Chai, Q. Ma, G. Wang, X. Wang, J. Su, J. Xu, M. Zhu, Y. Cheng, J. Yuan, J. Li, K. Kuang, Y. Yang, H. Yang, and F. Wu, "InfAgent-DABench: Evaluating agents on data analysis tasks," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. Vienna, Austria: JMLR.org, 2024.
- [12] L. Mitchener, J. M. Laurent, A. Andonian, B. Tenmann, S. Narayanan, G. P. Wellawatte, A. White, L. Sani, and S. G. Rodrigues, "BixBench: A Comprehensive Benchmark for LLM-based Agents in Computational Biology," 2025.
- [13] Y. Sui, H. Zhao, R. Ma, Z. He, H. Wang, J. Li, and Y. Yang, "Act While Thinking: Accelerating LLM Agents via Pattern-Aware Speculative Tool Execution," 2026.
- [14] Z. Pan, A. Patel, Z. Hu, Y. Shen, Y. Guan, W.-L. Li, L. Qin, Y. Wang, and Y. Ding, "KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows," 2025.
- [15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*. Koblenz Germany: ACM, Oct. 2023, pp. 611–626.
- [16] W. Shin, R. Souza, D. Rosendo, F. Suter, F. Wang, P. Balaprakash, and R. Ferreira Da Silva, "The (R)evolution of Scientific Workflows in the Agentic AI Era: Towards Autonomous Science," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. St Louis MO USA: ACM, Nov. 2025, pp. 2305–2316.
- [17] J. G. Pauloski, K. Chard, and I. Foster, "Agentic Discovery: Closing the Loop With Cooperative Agents," *Computer*, vol. 58, no. 10, pp. 20–27, Oct. 2025.
- [18] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Ding, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Chen, J. Yuan, J. Tu, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. You, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Zhou, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang, "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, no. 8081, pp. 633–638, Sep. 2025.
- [19] L. Jing, Z. Huang, X. Wang, W. Yao, W. Yu, K. Ma, H. Zhang, X. Du, and D. Yu, "DSBench: How Far Are Data Science Agents from Becoming Data Science Experts?" 2024.
- [20] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, L. Weng, and A. Madry, "MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering," 2024.
- [21] E. Lai, G. Vitagliano, Z. Zhang, O. Chabira, S. Sudhir, A. Zeng, A. A. Zabreyko, C. Li, F. Kossmann, J. Ding, J. Chen, M. Markakis, M. Russo, W. Wang, Z. Wu, M. J. Cafarella, L. Cao, S. Madden, and T. Kraska, "KramaBench: A Benchmark for AI Systems on Data-to-Insight Pipelines over Data Lakes," 2025.
- [22] P. Carns, J. Kunkel, K. Mohror, and M. Schulz, "Understanding I/O Behavior in Scientific and Data-Intensive Computing (Dagstuhl Seminar 21332)," Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Tech. Rep. 7, 2021.
- [23] T. Patel, S. Byna, G. K. Lockwood, N. J. Wright, P. Carns, R. Ross, and D. Tiwari, "Uncovering access, reuse, and sharing characteristics of I/O-Intensive files on Large-Scale production HPC systems," in *18th USENIX Conference on File and Storage Technologies (FAST 20)*. Santa Clara, CA: USENIX Association, Feb. 2020, pp. 91–101.
- [24] I. Yildirim, H. Devarajan, A. Kougkas, X.-H. Sun, and K. Mohror, "WisIO: Automated I/O Bottleneck Detection with Multi-Perspective Views for HPC Workflows," in *ICS '25: 2025 International Conference on Supercomputing*, Salt Lake City, UT, USA, Jun. 2025.
- [25] Y. Zheng, Y. Hu, T. Yu, and A. Quinn, "AgentSight: System-Level Observability for AI Agents Using eBPF," in *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*. Seoul Republic of Korea: ACM, Oct. 2025, pp. 110–115.
- [26] R. Souza, A. Gueroudji, S. DeWitt, D. Rosendo, T. Ghosal, R. Ross, P. Balaprakash, and R. F. Da Silva, "PROV-AGENT: Unified Provenance for Tracking AI Agent Interactions in Agentic Workflows," in *2025 IEEE International Conference on eScience (eScience)*. Chicago, IL, USA: IEEE, Sep. 2025, pp. 467–473.
- [27] "OpenTelemetry semantic conventions for generative AI systems," 2025.
- [28] "Langfuse observability documentation," 2026.
- [29] J. Yao, H. Li, Y. Liu, S. Ray, Y. Cheng, Q. Zhang, K. Du, S. Lu, and J. Jiang, "CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion," in *Proceedings of the Twentieth European Conference on Computer Systems*. Rotterdam Netherlands: ACM, Mar. 2025, pp. 94–109.
- [30] D. Joseph and D. Grunwald, "Prefetching using Markov predictors," in *Proceedings of the 24th Annual International Symposium on Computer Architecture*. Denver Colorado USA: ACM, May 1997, pp. 252–263.
- [31] N. Megiddo and D. S. Modha, "ARC: A Self-Tuning, low overhead replacement cache," in *2nd USENIX Conference on File and Storage Technologies (FAST 03)*. San Francisco, CA: USENIX Association, Mar. 2003.
- [32] C. Daley, D. Ghoshal, G. Lockwood, S. Dosanjh, L. Ramakrishnan, and N. Wright, "Performance characterization of scientific workflows for the

- optimal use of Burst Buffers,” *Future Generation Computer Systems*, vol. 110, pp. 468–480, Sep. 2020.
- [33] H. Li, B. Handsaker, A. Wysoker, T. Fennell, J. Ruan, N. Homer, G. Marth, G. Abecasis, R. Durbin, and 1000 Genome Project Data Processing Subgroup, “The Sequence Alignment/Map format and SAMtools,” *Bioinformatics*, vol. 25, no. 16, pp. 2078–2079, Aug. 2009.
- [34] C. Wang, I. Yildirim, H. Devarajan, K. Mohror, and M. Snir, “Recorder: Comprehensive Parallel I/O Tracing and Analysis,” 2025.
- [35] H. Devarajan, L. Pottier, K. Velusamy, H. Zheng, I. Yildirim, O. Kogiou, W. Yu, A. Kougkas, X.-H. Sun, J. S. Yeom, and K. Mohror, “DFTracer: An Analysis-Friendly Data Flow Tracer for AI-Driven Workflows,” in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. Atlanta, GA, USA: IEEE, Nov. 2024, pp. 1–24.
- [36] X. Hou, Y. Zhao, S. Wang, and H. Wang, “Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions,” *ACM Transactions on Software Engineering and Methodology*, p. 3796519, Feb. 2026.
- [37] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” 2022.
- [38] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” 2023.
- [39] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 68 539–68 551.
- [40] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for Transformer-Based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538.
- [41] C. Barrett, S. Cao, J. Gonzalez, J. Huang, C. Kozyrakis, Y. Sheng, I. Stoica, C. Sun, Z. Xie, L. Yin, C. Yu, and L. Zheng, “SGLang: Efficient Execution of Structured Language Model Programs,” in *Advances in Neural Information Processing Systems 37*. Vancouver, BC, Canada: Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024, pp. 62 557–62 583.
- [42] I. Giurgiu and M. E. Nidd, “Supporting Dynamic Agentic Workloads: How Data and Agents Interact,” 2025.
- [43] R. H. Patterson, G. A. Gibson, E. Ginting, D. Stodolsky, and J. Zelenka, “Informed prefetching and caching,” in *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles - SOSP ’95*. Copper Mountain, Colorado, United States: ACM Press, 1995, pp. 79–95.
- [44] A. Kougkas, H. Devarajan, and X.-H. Sun, “Hermes: A heterogeneous-aware multi-tiered distributed I/O buffering system,” in *Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing*. Tempe Arizona: ACM, Jun. 2018, pp. 219–230.
- [45] T. Wang, K. Mohror, A. Moody, K. Sato, and W. Yu, “An Ephemeral Burst-Buffer File System for Scientific Applications,” in *SC16: International Conference for High Performance Computing, Networking, Storage and Analysis*. Salt Lake City, UT, USA: IEEE, Nov. 2016, pp. 807–818.
- [46] M. J. Brim, A. T. Moody, S.-H. Lim, R. Miller, S. Boehm, C. Stanavige, K. M. Mohror, and S. Oral, “UnifyFS: A User-level Shared File System for Unified Access to Distributed Local Storage,” in *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. St. Petersburg, FL, USA: IEEE, May 2023, pp. 290–300.
- [47] M.-A. Vef, N. Moti, T. Süß, M. Tacke, T. Tocci, R. Nou, A. Miranda, T. Cortes, and A. Brinkmann, “GekkoFS — A Temporary Burst Buffer File System for HPC Applications,” *Journal of Computer Science and Technology*, vol. 35, no. 1, pp. 72–91, Jan. 2020.
- [48] J. G. Pauloski, V. Hayot-Sasson, L. Ward, N. Hudson, C. Sabino, M. Baughman, K. Chard, and I. Foster, “Accelerating Communications in Federated Applications with Transparent Object Proxies,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. Denver CO USA: ACM, Nov. 2023, pp. 1–15.
- [49] H. Devarajan, A. Kougkas, and X.-H. Sun, “HFatch: Hierarchical Data Prefetching for Scientific Workflows in Multi-Tiered Storage Environments,” in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. New Orleans, LA, USA: IEEE, May 2020, pp. 62–72.
- [50] Z. Liang, J. Lombardi, M. Chaarawi, and M. Hennecke, “DAOS: A Scale-Out High Performance Storage Stack for Storage Class Memory,” in *Supercomputing Frontiers*, D. K. Panda, Ed. Cham: Springer International Publishing, 2020, vol. 12082, pp. 40–54.
- [51] B. K. R. Vangoor, P. Agarwal, M. Mathew, A. Ramachandran, S. Sivaraman, V. Tarasov, and E. Zadok, “Performance and Resource Utilization of FUSE User-Space File Systems,” *ACM Transactions on Storage*, vol. 15, no. 2, pp. 1–49, May 2019.
- [52] J. Moore and S. Kunis, “Zarr: A Cloud-Optimized Storage for Interactive Access of Large Arrays,” *Proceedings of the Conference on Research Data Infrastructure*, vol. 1, Sep. 2023.
- [53] The 1000 Genomes Project Consortium, “A map of human genome variation from population-scale sequencing,” *Nature*, vol. 467, no. 7319, pp. 1061–1073, Oct. 2010.
- [54] B. Bode, G. Bauer, L. Herriott, V. Kindratenko, and W. Gropp, “DeltaAI: A National Resource for AI/ML Research,” in *Practice and Experience in Advanced Research Computing 2025: The Power of Collaboration*. Columbus Ohio USA: ACM, Jul. 2025, pp. 1–4.
- [55] T. J. Boerner, S. Deems, T. R. Furlani, S. L. Knuth, and J. Towns, “ACCESS: Advancing Innovation: NSF’s Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support,” in *Practice and Experience in Advanced Research Computing*. Portland OR USA: ACM, Jul. 2023, pp. 173–176.